

Makine Öğrenmesi Proje Dokümantasyonu — Deployment

AI for Food Waste Reduction · SIC AI 17 Capstone, Grup 11

Ekip: Osman Furkan Erkan, Nermin Kılıçarslan, Dilara Kuyar, Ahsen Nisa Sarıkaya

Görev: Kaggle Food Demand Forecasting verisiyle, her hazırlık merkezi (fulfilment center) ve yemek için haftalık sipariş talebini (num_orders) tahmin etmek. Tahminler bir güven aralığıyla verilir; böylece mutfaklar üretimi talebe göre planlayıp gıda israfını azaltabilir.

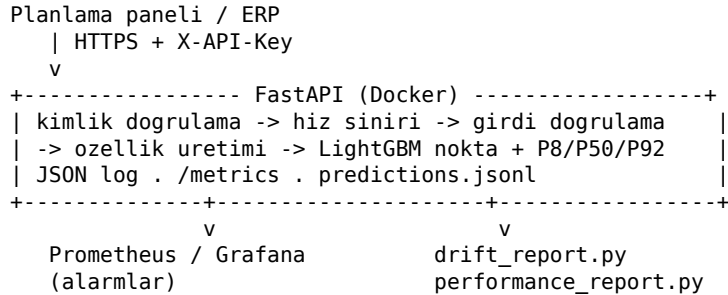
Deploy edilen model: LightGBM (bir nokta tahmin modeli + üç quantile modeli), 1-145. haftalar üzerinde (456.548 satır) eğitildi.

Modelleme aşamasında raporlanan sonuç	Değer
Holdout RMSLE (ayarlanmış model)	0,47534
Recursive walk-forward RMSLE (3 fold ortalaması)	0,49833
Persistence baseline RMSLE	0,80930
Persistence baseline'a göre iyileşme	%38,3
Kalibre %80 tahmin bandı kapsamı (coverage)	%80,59

1. Genel Bakış

Deployment aşaması, eğitilmiş LightGBM modellerini bir planlama paneli veya ERP sisteminin HTTPS üzerinden çağırabileceği bir servise dönüştürür. Uygulanan adımlar:

- Artefakt hazırlama.** deploy/scripts/export_artifacts.py dört üretim modelini toplar, pickle formatındaki label encoder'ları JSON kategori haritasına çevirir ve her model dosyası için özellik listesi, hedef dönüşümü ve SHA-256 özetini içeren bir manifest.json yazar.
- API ile sunum.** FastAPI uygulaması (deploy/app/main.py) artefaktları açılışa bir kez yükler, eğitim pipeline'ındaki (extensions/forecasting/pipeline.py) özellikleri birebir aynı şekilde yeniden üretir ve nokta tahmin ile %80'lik aralık döndürür.
- Test.** 9 otomatik test (deploy/tests/test_api.py) gerçek model dosyalarıyla çalışır; API'nin ürettiği özelliklerin notebook'taki özelliklerle aynı olduğu da kontrol edilir.
- Konteynerleştirme.** Dockerfile servisi paketler (root olmayan kullanıcı, health check).
- Güvenlik.** API anahtarı ile kimlik doğrulama, anahtar başına hız sınırı, sıkı girdi doğrulama, platform seviyesinde HTTPS, model bütünlük kontrolü.
- İzleme.** Yapılandırılmış JSON log, Prometheus metrikleri, tahmin kaydı, PSI tabanlı drift raporu, gerçek siparişlerle karşılaştıran performans raporu ve alarm kuralları.



2. Model Serileştirme (Serialization)

Format. LightGBM'in kendi metin formatı (Booster.save_model → .txt). İşletim sistemleri arasında taşınabilir, okunabilir ve pickle'ın aksine yüklenirken kod çalıştıramaz. Dört model servis edilir:

Rol	Dosya	Boyut	Not
Nokta tahmin	final_submission_lgb_model.txt	11,4 MB	1.360 tur, Optuna ile ayarlı parametreler
Alt sınır	quantile_final_p8.txt	8,0 MB	alpha = 0,08 · 969 tur
Medyan	quantile_final_p50.txt	15,9 MB	alpha = 0,50 · 1.925 tur
Üst sınır	quantile_final_p92.txt	8,0 MB	alpha = 0,92 · 951 tur

Model dosyasının içinde olmayanlar ve çözümü

Bir LightGBM modeli yalnızca ağaçları içerir. Tahmin sırasında iki şeye daha ihtiyaç vardır:

- **Hedef dönüşümü.** Modeller log1p(num_orders) üzerinde eğitildi; bu yüzden her tahmin expm1 ile geri çevrilir ve 0'da kesilir (pipeline.recursive_predict ile aynı).
- **Kategorik kodlamalar.** center_type, category ve cuisine sıralı benzersiz değerlerle label-encode edildi. export_artifacts.py, data/label_encoders.pkl dosyasını models/category_maps.json'a çevirir. Bu, pickle bağımlılığını kaldırır (güvenlik ve scikit-learn sürüm uyumluluğu).

```
# deploy/scripts/export_artifacts.py (özet)
encoders = pickle.load(open(SRC / "label_encoders.pkl", "rb"))
category_maps = {k: {str(c): i for i, c in enumerate(v.classes_)}}
                  for k, v in encoders.items()}
manifest["models"][key] = {"file": name, "sha256": sha256(DST / name), ...}
```

Depolama ve verimlilik değerlendirmeleri

Konu	Karar
Bütünlük	Her dosya için manifest'te SHA-256; özet uyuşmazsa API başlamayı reddeder
Sürümleme	API model_version = <api sürümü>+<nokta modelin özetinin ilk 8 karakteri> döndürür, örn. 1.0.0+7011fe06
Yükleme süresi	Modeller istek başına değil, açılışta bir kez yüklenir; toplam ≈ 43 MB küçük bir konteynere rahatça sığar
Repo boyutu	data/ klasörü Git'te ≈ 81 MB. Üretimde modelleri commit'lemek yerine Git LFS, DVC veya nesne depolama kullanılmalı
Tekrarlanabilirlik	Özellik listesi, hedef dönüşümü ve eğitim haftası aralığı manifest'te; kütüphane sürümleri requirements.txt'te sabitlenmiş

3. Model Sunumu (Serving)

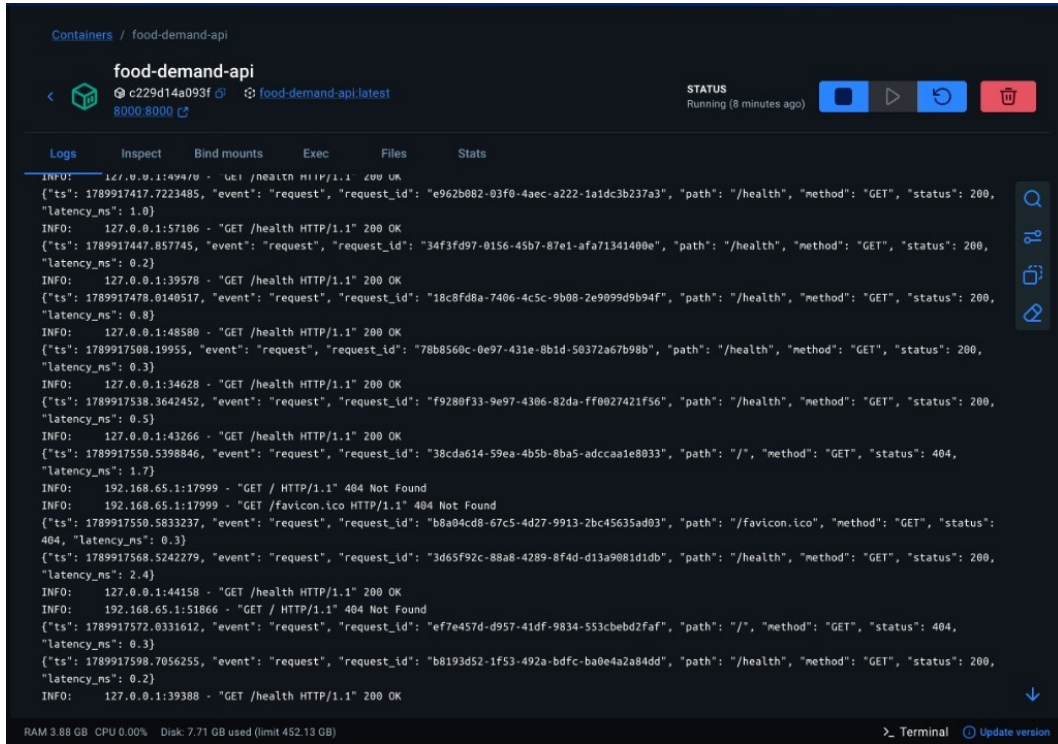
Desen. REST API arkasında çevrimiçi (online) çıkarım, durumsuz (stateless): istemci, o haftanın girdilerini ve ilgili merkez-yemek çiftinin yakın geçmişteki gerçek sipariş değerlerini gönderir. Bu, servisi yatay ölçeklemeyi kolaylaştırır; temel sürüm için veritabanı gerekmez.

Geçmiş neden istekte gönderiliyor? Modelin en önemli üç talep özelliği hedefin gecikmeleridir (num_orders_lag_1, num_orders_lag_4, num_orders_roll_mean_4). Eğitimde bunlar veri setinden gelir; üretimde müşterinin kendi satış geçmişinden gelmelidir. İstemci recent_orders listesini (eskiden yeniye) gönderir ve API gecikmeleri pipeline.add_history_features ile aynı kurallarla hesaplar:

Özellik	Kural
num_orders_lag_1	Listenin son elemanı; yoksa NaN
num_orders_lag_4	≥ 4 değer varsa sondan 4. eleman; yoksa NaN
num_orders_roll_mean_4	≥ 4 değer varsa son 4 değerın ortalaması; yoksa NaN

4 haftadan az geçmiş verilirse (yeni yemek veya merkez — cold start) özellikler NaN olur, LightGBM bunu doğal olarak işler ve belirsizlik daha yüksek olduğu için yanıtı cold_start uyarısı eklenir.

Çalışma ortamı. Docker içinde uvicorn (python:3.12-slim + LightGBM için libgomp1). Yerelde, ağ olmadan ölçülen süreler: tek tahmin (4 model) ≈ 7 ms, 500 satırlık batch $\approx 0,4$ sn.



```
Containers / food-demand-api
food-demand-api
c229d14a093f food-demand-api:latest
8000.8000

STATUS
Running (8 minutes ago)

Logs Inspect Bind mounts Exec Files Stats
INFO: 127.0.0.1:49470 - GET /health HTTP/1.1 200 OK
{"ts": 1789917417.7223485, "event": "request", "request_id": "e962b082-03f0-4aec-a222-1a1dc3b237a3", "path": "/health", "method": "GET", "status": 200, "latency_ms": 1.0}
INFO: 127.0.0.1:57106 - GET /health HTTP/1.1 200 OK
{"ts": 1789917447.857745, "event": "request", "request_id": "34f3fd97-0156-45b7-87e1-afa71341400e", "path": "/health", "method": "GET", "status": 200, "latency_ms": 0.2}
INFO: 127.0.0.1:39578 - GET /health HTTP/1.1 200 OK
{"ts": 1789917478.0140517, "event": "request", "request_id": "18c8fd8a-7406-4c5c-9b08-2e9099d9b94f", "path": "/health", "method": "GET", "status": 200, "latency_ms": 0.8}
INFO: 127.0.0.1:48580 - GET /health HTTP/1.1 200 OK
{"ts": 1789917508.19955, "event": "request", "request_id": "78b8560c-0e97-431e-8b1d-50372a67b98b", "path": "/health", "method": "GET", "status": 200, "latency_ms": 0.3}
INFO: 127.0.0.1:34628 - GET /health HTTP/1.1 200 OK
{"ts": 1789917538.3642452, "event": "request", "request_id": "f9280f33-9e97-4306-82da-ff0027421f56", "path": "/health", "method": "GET", "status": 200, "latency_ms": 0.5}
INFO: 127.0.0.1:43266 - GET /health HTTP/1.1 200 OK
{"ts": 1789917550.5398846, "event": "request", "request_id": "38cda614-59ea-4b5b-8ba5-adccaa1e8033", "path": "/", "method": "GET", "status": 404, "latency_ms": 1.7}
INFO: 192.168.65.1:17999 - GET / HTTP/1.1 404 Not Found
INFO: 192.168.65.1:17999 - GET /favicon.ico HTTP/1.1 404 Not Found
{"ts": 1789917550.5833237, "event": "request", "request_id": "b8a04cd8-67c5-4d27-9913-2bc45635ad03", "path": "/favicon.ico", "method": "GET", "status": 404, "latency_ms": 0.3}
{"ts": 1789917588.5242279, "event": "request", "request_id": "3d65f92c-8ba8-4289-8f4d-d13a9081d1db", "path": "/health", "method": "GET", "status": 200, "latency_ms": 2.4}
INFO: 127.0.0.1:44158 - GET /health HTTP/1.1 200 OK
INFO: 192.168.65.1:51866 - GET / HTTP/1.1 404 Not Found
{"ts": 1789917572.0331612, "event": "request", "request_id": "ef7e457d-d957-41df-9834-553cbebd2faf", "path": "/", "method": "GET", "status": 404, "latency_ms": 0.3}
{"ts": 1789917598.7056255, "event": "request", "request_id": "b8193d52-1f53-492a-bdfc-ba0e4a2a84dd", "path": "/health", "method": "GET", "status": 200, "latency_ms": 0.2}
INFO: 127.0.0.1:39388 - GET /health HTTP/1.1 200 OK

RAM 3.88 GB CPU 0.00% Disk 7.71 GB used (limit 452.13 GB)
Terminal Update version
```

Şekil 1 — Docker Desktop'ta çalışan food-demand-api konteyneri (image: food-demand-api:latest, port 8000:8000, durum: Running) ve yapılandırılmış JSON istek logları (request_id, path, method, status, latency_ms). Bu ekran görüntüsü, servisin yalnızca kod olarak değil, belgede tarif edilen konteyner olarak da gerçekten çalıştığını doğrular.

Platform seçenekleri

Seçenek	Artıları	Eksileri	Uygunluk
Render / Railway (Docker web service)	GitHub'dan dakikalar içinde deploy, ücretsiz HTTPS, secret deposu	Ücretsiz katmanda uyku modu (cold start)	Capstone demosu için uygun
Hugging Face Spaces (Docker)	Ücretsiz, paylaşması kolay	Varsayılan olarak herkese açık; API anahtarı Space secret olmalı	Alternatif demo
Google Cloud Run / AWS App Runner / Azure Container Apps	Otomatik ölçekleme (sıfıra kadar), yönetilen TLS, IAM, loglama	Kurulum ve maliyet	Üretim için uygun
Şirket içi (on-premises) sunucu	Tam veri kontrolü	Yama, ölçekleme ve erişilebilirlik bizde	Yalnızca müşteri isterse

Deploy adımları (Render örneği). Repoyu push'la → New Web Service → kök dizin deploy/ → runtime Docker → API_KEYS, ALLOWED_ORIGINS, RATE_LIMIT_PER_MIN ortam değişkenlerini gir → deploy. main'e her push, CI workflow'u (deploy-ci.yml: artefakt üret → pytest → bağımlılık taraması → docker build) ile yeniden deploy edilir.

Çok haftalık tahmin. Kaggle submission'ı 10 hafta ilerisini recursive tahmin eder (her haftanın tahmini bir sonraki haftanın geçmişi olur). API tek adımlık tahmin verir; daha uzun ufuk isteyen istemci API'yi hafta hafta çağırıp önceki tahmini recent_orders'a ekler (recursive_predict'in aynısı). Sunucu tarafı /forecast endpoint'i gelecek çalışma olarak 7. bölümde.

4. API Entegrasyonu

Etkileşimli dokümantasyon (Swagger UI) /docs adresinde otomatik oluşturulur.

Metot	Yol	Amaç	Kimlik
GET	/health	Canlılık kontrolü, yüklü modeller	-
GET	/model-info	Özellik listesi, kategoriler, eğitim aralığı, raporlanan metrikler	✓
POST	/predict	Tek merkez-yemek-hafta için tahmin	✓
POST	/predict/batch	Tek çağrıda en fazla 500 tahmin	✓
GET	/metrics	Prometheus metrikleri	✓

Girdi (POST /predict)

Alan	Tip / aralık	Açıklama
week	int 1-520	Tahmin edilecek hafta

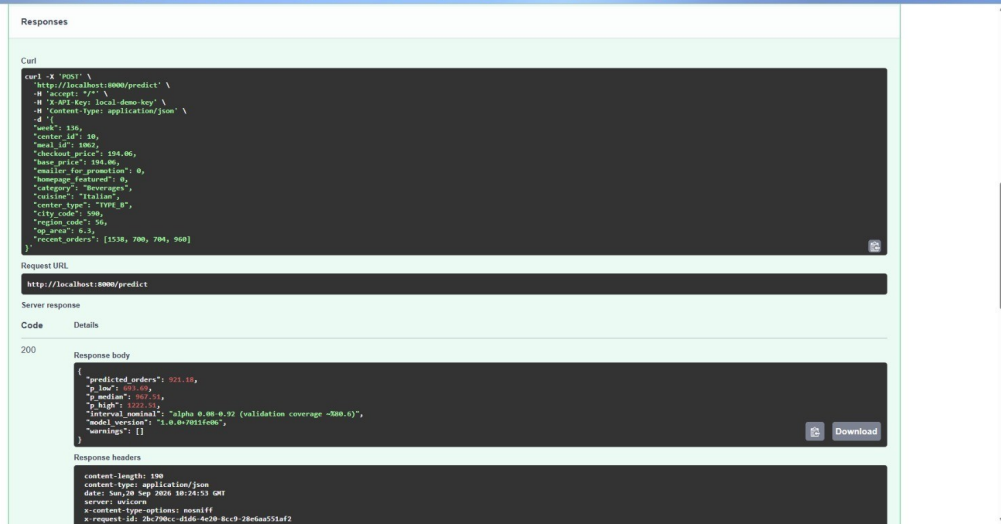
Alan	Tip / aralık	Açıklama
center_id, meal_id	int $\geq$ 0	Tanımlayıcılar
checkout_price, base_price	float > 0	İndirim sonrası / öncesi fiyat
emailer_for_promotion, homepage_featured	0 veya 1	Promosyon bayrakları
category	14 bilinen kategoriden biri	örn. Beverages, Pizza, Rice Bowl
cuisine	Continental, Indian, Italian, Thai	Mutfak türü
center_type	TYPE_A, TYPE_B, TYPE_C	Merkez tipi
city_code, region_code	int $\geq$ 0	Merkez konum kodları
op_area	float > 0	Merkezin operasyon alanı
recent_orders	$\leq$ 13 adet float $\geq$ 0	Gerçek geçmiş siparişler, eskiden yeniye

API içinde türetilenler: discount_ratio, weekofyear, week_sin, week_cos, kodlanmış kategorikler ve üç gecikme özelliği.

Örnek (deploy edilen modellerin gerçek çıktısı)

```
curl -X POST https://<servis>.onrender.com/predict \
-H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
-d '{
  "week":136,"center_id":10,"meal_id":1062,"checkout_price":194.06,
  "base_price":194.06,"emailer_for_promotion":0,"homepage_featured":0,
  "category":"Beverages","cuisine":"Italian","center_type":"TYPE_B",
  "city_code":590,"region_code":56,"op_area":6.3,
  "recent_orders":[1538,700,704,960]}'

{
  "predicted_orders": 921.18,
  "p_low": 693.69,
  "p_median": 967.51,
  "p_high": 1222.51,
  "interval_nominal": "alpha 0.08-0.92 (validation coverage ~%80.6)",
  "model_version": "1.0.0+7011fe06",
  "warnings": []
}
```



Şekil 2 — Swagger UI (/docs) üzerinden gönderilen gerçek POST /predict isteği ve sunucu yanıtı. $predicted_orders=921.18$, $p_low=693.69$, $p_median=967.51$, $p_high=1222.51$ değerleri, yukarıdaki curl örneğiyle ve bu raporun geri kalanında referans verilen sonuçlarla birebir aynıdır; yani örnek uydurma değil, çalışan servisten alınmıştır.

Bu satır için veri setindeki gerçek sipariş sayısı 918 idi: gerçek değer aralığının içinde ve nokta tahmin 3 sipariş sapmalı. recent_orders boş gönderildiğinde aynı istek predicted_orders = 320,19, çok daha geniş bir bant (166–771) ve cold_start uyarısı döndürür; geçmişin neden önemli olduğunu gösterir.

Çıktı israfı azaltmak için nasıl kullanılır? p_median beklenen talebi, p_high güvenli üst sınırı gösterir. Talebin üstünde üretim israfa, altında üretim kaçan satışa yol açar; israfı azaltmak isteyen mutfak medyana yakın planlar, p_high 'ı yalnızca stoksız kalma maliyeti yüksek ürünlerde kullanır.

Yanıt kodları

Kod	Anlamı
200	Başarılı
401	X-API-Key eksik veya geçersiz
413	Batch 500 satırdan büyük
422	Doğrulama hatası (aralık dışı değer, bilinmeyen kategori, negatif geçmiş)
429	Hız sınırı aşıldı (Retry-After: 60)
503	Sunucuda API anahtarı tanımlı değil
500	Beklenmeyen hata — istemciye genel mesaj, ayrıntı X-Request-ID ile logda

Test. pytest gerçek model dosyalarıyla 9 test çalıştırır: kimlik doğrulama, özelliklerin notebook ile eşitliği (gerçek bir veri satırının lag / rolling / döngüsel / kodlanmış değerleri), sıralı tahmin bandı ($p_low \leq p_median \leq p_high$), makul çıktı aralığı, cold-start uyarısı, girdi doğrulama, batch sınırları ve korumalı metrics endpoint'i.

5. Güvenlik Değerlendirmeleri

Katman	Önlem	Nerede
Kimlik doğrulama	X-API-Key başlığı, secrets.compare_digest ile (sabit zamanlı, zamanlama saldırılarına dayanıklı). Anahtar tanımlı değilse servis açık çalışmak yerine 503 döner	verify_key
Yetkilendirme	/health herkese açık; tahmin, model-info ve metrics anahtar ister. İstemci başına ayrı anahtar verilebilir ve API_KEYS düzenlenerek tek tek iptal edilir	ortam değişkeni
Aktarımda şifreleme	TLS platform tarafından sonlandırılır (Render, Cloud Run ...); HTTP, HTTPS'e yönlendirilir	hosting
Beklemede şifreleme	Yönetilen platform diskleri / secret depoları; kişisel veri saklanmaz	hosting
Girdi doğrulama	Her alanda Pydantic aralık kontrolü, kategori beyaz listesi, sonlu ve negatif olmayan geçmiş, en fazla 13 geçmiş değeri, en fazla 500 batch satırı	DemandRequest
Kötüye kullanım koruması	Anahtar başına kayan pencere hız sınırı (varsayılan 120/dk) → 429; başarısız kimlik denemeleri sayılır ve ani artışta alarm verir	verify_key, alerts.yml
Model bütünlüğü	Açılıştan SHA-256 doğrulaması; modeller pickle'dan değil metin formatından yüklenir	load_artifacts
Gizli bilgiler	Anahtarlar yalnızca ortam değişkenleriyle; .env git'e eklenmez, .env.example değişkenleri belgeler	.env.example
CORS	ALLOWED_ORIGINS verilmedikçe kapalı; yalnızca GET/POST ve iki gerekli başlık	CORSMiddleware
Konteyner sıkılaştırma	Slim imaj, root olmayan kullanıcı, health check, .dockerignore ile test/log/secret hariç tutulur	Dockerfile
Tedarik zinciri	Sürüm aralıkları sabit; CI'da pip-audit	deploy-ci.yml
Hata hijyeni	Stack trace yalnızca sunucu logunda; istemciye genel 500 + istek kimliği; X-Content-Type-Options: nosniff	middleware
Gizlilik	İstekler yalnızca iş verisi içerir	tasarım

Katman	Önlem	Nerede
	(merkez, yemek, fiyat, sipariş); müşteri veya kişisel veri yok	

Bilinen sınırlar. Hız sınırlayıcı bellek içidir; birden fazla örnekte her biri kendi sayacını tutar — üretimde bir API gateway veya Redis tabanlı sınırlayıcı gerekir. API anahtarları statiktir; çok kiracılı kullanımda OAuth2/JWT gerekir.

6. İzleme ve Loglama (Monitoring and Logging)

6.1 Loglama

- **Uygulama logu:** her istek için tek satır JSON — request_id, yol, metot, durum, gecikme. Yakalanmamış hatalar aynı request_id ile stack trace'li loglanır.
- **Tahmin logu (logs/predictions.jsonl):** her tahmin; girdileri, n_history, model sürümü, nokta tahmin ve bandıyla kaydedilir. Amacı sonradan gerçek satışlarla birleştirilme. Loglama hatası tahmini engellemez.

6.2 İzlenen metrikler

Kategori	Metrik	Kaynak
Servis sağlığı	İstek hızı, hata oranı (5xx), yol başına p50/p95 gecikme	api_requests_total, api_request_latency_seconds (Prometheus)
Güvenlik	Geçersiz anahtar denemeleri, hız sınırına takılan istekler	auth_failures_total, rate_limited_total
Veri kalitesi	Cold-start isteklerinin payı	prediction_cold_start_total; drift raporunda cold_start_share
Veri kayması (drift)	checkout_price, base_price, discount_ratio ve promosyon bayraklarının referans veriye göre PSI değeri	monitoring/drift_report.py
Tahmin kayması	Tahmin edilen sipariş dağılımı (histogram + PSI)	prediction_num_orders, drift raporu
Model performansı	RMSLE, MAE, yanlılık (bias), bant kapsamı; cold-start ve sıcak istekler için ayrı RMSLE	monitoring/performance_report.py (tahmin logunu gerçek siparişlerle birleştirir)

Eşikler neden böyle? Performans raporu, RMSLE 0,548'i (ölcüğümüz 0,498 walk-forward RMSLE'nin %10 üstü) aştığında veya %80'lik bandın gözlenen kapsamı %70-90 aralığının dışına çıktığında alarm verir (kalibre kapsama %80,6 idi). PSI için olağan kural: < 0,10 stabil, 0,10-0,25 orta, > 0,25 ciddi kayma.

Doğrulama. Drift betiği gerçek holdout verisiyle denendi: değişmemiş veri → OK (PSI ≈ 0,005); simüle edilen %40 fiyat indirimi → ALERT (checkout_price için PSI 3,04). Performans betiği aynı örnekte RMSLE 0,454 ve kapsama %75,7 verdi → OK.

6.3 Alarmlar

Alarm	Koşul	Önem
ApiDown	Scrape hedefi 2 dakika kapalı	kritik
HighErrorRate	5 dakika boyunca 5xx payı > %2	kritik
SlowResponses	10 dakika boyunca p95 gecikme > 1 sn	uyarı
AuthFailureSpike	5 dakika boyunca saniyede > 0,5 geçersiz anahtar denemesi	uyarı
HighColdStartShare	1 saat boyunca tahminlerin > %30'u geçmişsiz	uyarı
Veri kayması	Herhangi bir PSI > 0,25 (betik çıkış kodu 1 → CI/cron bildirimi)	uyarı
Performans düşüşü	RMSLE > 0,548 veya kapsama %70-90 dışında	kritik

Kurallar monitoring/alerts.yml içindedir (Prometheus). Bildirim kanalı (e-posta/Slack) Alertmanager'da veya iki rapor betiğini zamanlanmış çalıştıran GitHub Actions içinde tanımlanır. Ücretsiz bir tamamlayıcı olarak /health için harici uptime kontrolü (örn. UptimeRobot) kullanılabilir.

6.4 Yeniden eğitim ve geri alma

Haftalık: tahmin logunu gerçek siparişlerle birleştir → iki raporu çalıştır. Alarm gelirse: genişletilmiş geçmişle extensions/forecasting/build_submission.py ile yeniden eğit, walk-forward protokolüyle mevcut modelle kıyasla, yeni artefaktları üret (yeni özet → yeni model_version) ve CI ile yeniden deploy et. Önceki imaj etiketi tek adımda geri dönüş için saklanır.